Airlines Reservation System

Phase 1
Conceptual Design

Phase 2
Logical Design

Phase 3
Physical Design

STUDENT INFORMATION

- **Student Name** : Mohammed Nairoukh
- **Student ID** : 2410205

- **Course Name** : Database Fundamentals
- **Instructor** : Dr. Omar Nobani

PHASE 1 · CONCEPTUAL DESIGN · ERD · Entities · Relationships

1.1 Project Description

The **Airlines Reservation System** is a comprehensive relational database designed to manage the core operations of a commercial airline company. It covers the complete lifecycle of air travel — from passenger registration and seat booking, through flight scheduling and airport operations, to payment processing and crew management.

The system stores each flight's unique identifier, departure/arrival times, and operational status. Passengers are recorded with personal details including passport number and contact information. A reservation entity links passengers to specific flights and captures booking metadata such as date and status. Tickets represent individual seats assigned to reservations and carry class and price information. Airports are defined by name and location, while aircraft models include capacity and airline ownership. The system additionally tracks crew members (staff roles), baggage details, and recurring flight schedules. Integrity constraints prevent overlapping schedules and enforce valid relational references throughout.

1.2 Business Rules & Assumptions

Entity / Concept	Business Rule
Passenger	Each passenger has a unique Passenger_ID and a unique Passport_No. A passenger may make many reservations over time.
Reservation	A reservation is made by exactly one passenger and can include multiple tickets (one per seat). Status values: Confirmed, Pending, Cancelled.
Ticket	Each ticket belongs to exactly one reservation and is assigned to exactly one seat. It records class (Economy, Business, First) and price.
Seat	A seat belongs to a specific flight (via the Ticket relationship). Seat numbers are unique per flight. A seat is used by at most one ticket.
Flight	Each flight is uniquely identified by Flight_ID and a human-readable Flight_Number. It operates between two airports (departure & arrival) and is owned by one airline.
Airport	Each airport has a unique Airport_ID, a name, and a location (city/country). A flight can operate at many airports (departure and arrival).
Airline	An airline owns many flights. Each flight belongs to exactly one airline.
Staff	Staff members can be pilots, cabin crew, or ground staff. Many staff members may be assigned to manage flights (M:1).
Schedule	A flight can have one recurring schedule (day + time). Schedule_ID uniquely identifies each schedule entry.
Baggage	Baggage records are associated with a passenger (recursive relationship). Each baggage item has weight and type.
Payment	Payments are made by passengers. A passenger may have multiple payment records. Payment stores amount, date, and method.

1.3 Entity-Relationship Diagram (ERD)

The ERD below was designed following **Chen notation**: rectangles represent entities, ovals represent attributes (underlined = primary key), diamonds represent relationships, and lines show cardinality (1 or M).

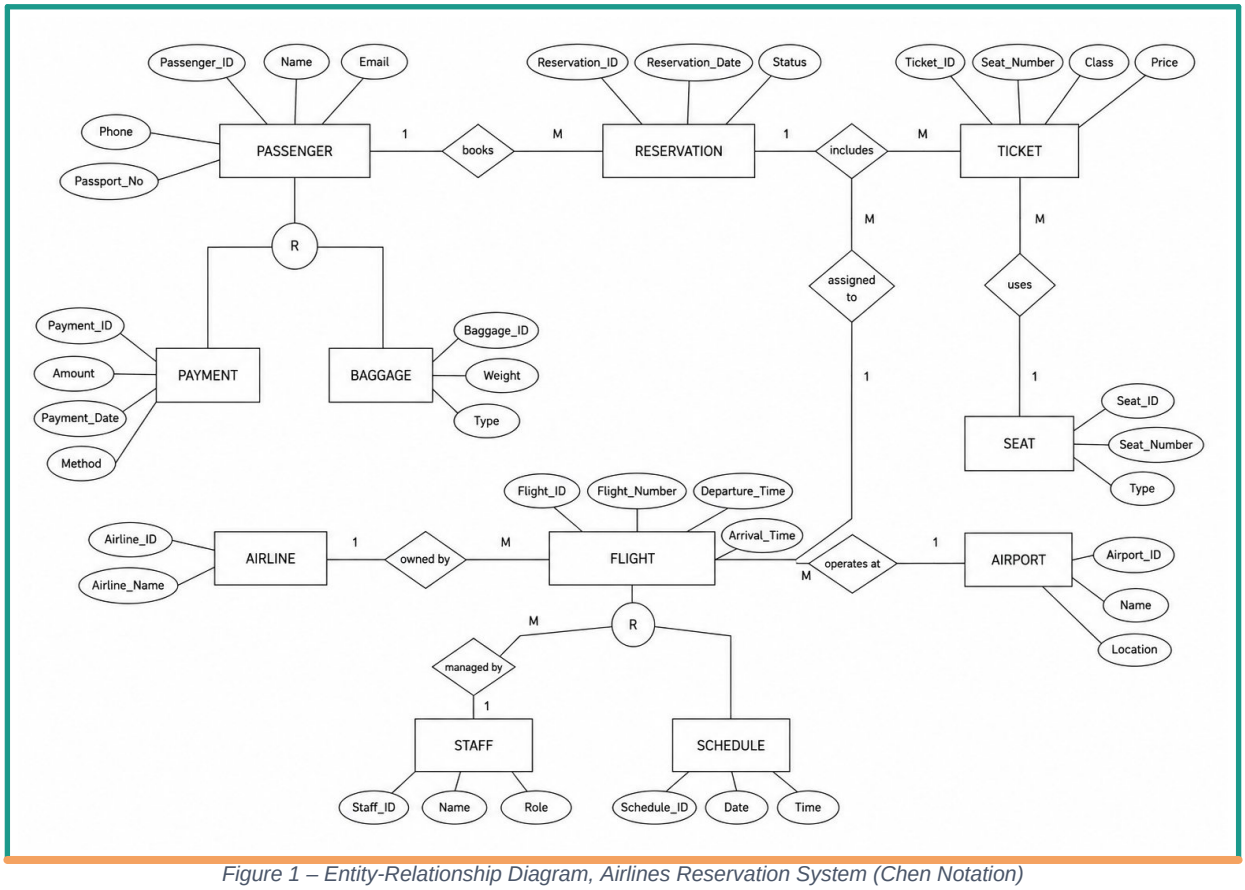

Figure 1 – Entity-Relationship Diagram, Airlines Reservation System (Chen Notation)

■ PASSENGER
Passenger_ID (PK) · Name · Email · Phone · Passport_No

■ RESERVATION
Reservation_ID (PK) · Reservation_Date · Status

■ TICKET
Ticket_ID (PK) · Seat_Number · Class · Price

■ SEAT
Seat_ID (PK) · Seat_Number · Type

■ FLIGHT
Flight_ID (PK) · Flight_Number · Departure_Time · Arrival_Time

■ AIRPORT
Airport_ID (PK) · Name · Location

■ AIRLINE
Airline_ID (PK) · Airline_Name

■ STAFF
Staff_ID (PK) · Name · Role

■ SCHEDULE
Schedule_ID (PK) · Date · Time

■ BAGGAGE
Baggage_ID (PK) · Weight · Type

■ PAYMENT
Payment_ID (PK) · Amount · Payment_Date · Method

1.4 Entities & Attributes

Entity	Attributes
PASSENGER	Passenger_ID (PK), Name, Email, Phone, Passport_No
RESERVATION	Reservation_ID (PK), Reservation_Date, Status
TICKET	Ticket_ID (PK), Seat_Number, Class, Price
SEAT	Seat_ID (PK), Seat_Number, Type
FLIGHT	Flight_ID (PK), Flight_Number, Departure_Time, Arrival_Time
AIRPORT	Airport_ID (PK), Name, Location
AIRLINE	Airline_ID (PK), Airline_Name
STAFF	Staff_ID (PK), Name, Role
SCHEDULE	Schedule_ID (PK), Date, Time
BAGGAGE	Baggage_ID (PK), Weight, Type
PAYMENT	Payment_ID (PK), Amount, Payment_Date, Method

1.5 Relationships & Cardinalities

Relationship	Entities	Cardinality	Description
books	PASSENGER → RESERVATION	1 : M	One passenger books many reservations
includes	RESERVATION → TICKET	1 : M	One reservation includes many tickets
assigned to	TICKET → SEAT	M : 1	Many tickets assigned to one seat
uses	TICKET → SEAT	M : 1	Ticket uses a specific seat
owned by	FLIGHT → AIRLINE	M : 1	Many flights owned by one airline
operates at	FLIGHT → AIRPORT	M : 1	Flight operates at one airport (departure/arrival)
managed by	FLIGHT → STAFF	M : 1	Many flights managed by one staff member
R (recursive)	PASSENGER → BAGGAGE	1 : M	Passenger has many baggage items
R (recursive)	FLIGHT → SCHEDULE	1 : M	Flight has many schedule entries

PHASE 2 · LOGICAL DESIGN & NORMALIZATION · Relational Schema · 3NF

2.1 Relational Schema

The ERD is mapped to the following relational schemas. **Primary keys are bold and underlined**; foreign keys are marked with (FK).

Table	Attributes	Foreign Keys
PASSENGER	Passenger_ID (PK), Name, Email, Phone, Passport_No	—
PAYMENT	Payment_ID (PK), Amount, Payment_Date, Method, Passenger_ID (FK→ PASSENGER)	Passenger_ID
BAGGAGE	Baggage_ID (PK), Weight, Type, Passenger_ID (FK → PASSENGER)	Passenger_ID
RESERVATION	Reservation_ID (PK), Reservation_Date, Status, Passenger_ID (FK → PASSENGER)	Passenger_ID
AIRLINE	Airline_ID (PK), Airline_Name	—
AIRPORT	Airport_ID (PK), Name, Location	—
STAFF	Staff_ID (PK), Name, Role	—
FLIGHT	Flight_ID (PK), Flight_Number, Departure_Time, Arrival_Time, Airline_ID (FK→ AIRLINE), Airport_ID (FK→ AIRPORT), Staff_ID (FK→ STAFF)	Airline_ID, Airport_ID, Staff_ID
SCHEDULE	Schedule_ID (PK), Date, Time, Flight_ID (FK → FLIGHT)	Flight_ID
SEAT	Seat_ID (PK), Seat_Number, Type, Flight_ID (FK → FLIGHT)	Flight_ID
TICKET	Ticket_ID (PK), Seat_Number, Class, Price, Reservation_ID (FK→ RESERVATION), Seat_ID (FK→ SEAT)	Reservation_ID, Seat_ID

2.2 Normalization Assessment (3NF)

Each table is examined against 1NF, 2NF, and 3NF. Functional dependencies (FDs) are listed and any decompositions required to achieve 3NF are explained.

Table	Functional Dependency	1NF	2NF	3NF	Justification
PASSENGER	Passenger_ID → Name, Email, Phone, Passport_No	✔	✔	✔	No partial or transitive dependencies. All non-key attrs depend solely on Passenger_ID.
RESERVATION	Reservation_ID → Reservation_Date, Status, Passenger_ID	✔	✔	✔	Single-attribute PK eliminates partial deps. Status is a direct property of the reservation.
TICKET	Ticket_ID → Seat_Number, Class, Price, Reservation_ID, Seat_ID	✔	✔	✔	All attributes depend fully on Ticket_ID. Price is ticket-level, not seat-class level.
SEAT	Seat_ID → Seat_Number, Type, Flight_ID	✔	✔	✔	Seat_Number unique per flight is a business rule enforced by constraint, not a FD issue.

Table	Functional Dependency	1NF	2NF	3NF	Justification
FLIGHT	Flight_ID → Flight_Number, Departure_Time, Arrival_Time, Airline_ID, Airport_ID, Staff_ID	✔	✔	✔	No non-key attribute determines another non-key attribute.
SCHEDULE	Schedule_ID → Date, Time, Flight_ID	✔	✔	✔	Single PK; Date+Time describe the schedule entry, not derived from each other.
AIRLINE	Airline_ID → Airline_Name	✔	✔	✔	Trivially in 3NF.
AIRPORT	Airport_ID → Name, Location	✔	✔	✔	Trivially in 3NF.
STAFF	Staff_ID → Name, Role	✔	✔	✔	Trivially in 3NF.
PAYMENT	Payment_ID → Amount, Payment_Date, Method, Passenger_ID	✔	✔	✔	No transitive deps; Method is payment-level attribute.
BAGGAGE	Baggage_ID → Weight, Type, Passenger_ID	✔	✔	✔	Trivially in 3NF.

Conclusion: All 11 tables satisfy Third Normal Form (3NF). No decomposition was required because the schema was designed with single-attribute primary keys that fully determine all non-key attributes, and no non-key attribute transitively determines another.

PHASE 3 · IMPLEMENTATION & QUERYING · DDL · DML · Analytical SQL

3.1 Data Definition Language (DDL)

The following script creates the database and all tables with primary keys, foreign keys, NOT NULL, UNIQUE, and CHECK constraints. Compatible with **SQL Server, MySQL, and PostgreSQL**.

```
-- 1. PASSENGER
CREATE TABLE PASSENGER (
    Passenger_ID  INT          PRIMARY KEY,
    Name          VARCHAR(100) NOT NULL,
    Email         VARCHAR(150) NOT NULL UNIQUE,
    Phone         VARCHAR(20),
    Passport_No   VARCHAR(20)  NOT NULL UNIQUE
);
```

```
-- 2. PAYMENT
CREATE TABLE PAYMENT (
    Payment_ID    INT            PRIMARY KEY,
    Amount        DECIMAL(10,2)  NOT NULL CHECK (Amount > 0),
    Payment_Date  DATE           NOT NULL,
    Method        VARCHAR(50)    NOT NULL
        CHECK (Method IN ('Credit Card','Debit Card','Cash','Online')),
    Passenger_ID  INT            NOT NULL
        REFERENCES PASSENGER(Passenger_ID)
);
```

```
-- 3. BAGGAGE
CREATE TABLE BAGGAGE (
    Baggage_ID    INT           PRIMARY KEY,
    Weight        DECIMAL(5,2)  NOT NULL CHECK (Weight > 0),
    Type          VARCHAR(50)   NOT NULL
        CHECK (Type IN ('Carry-On','Checked','Oversized')),
    Passenger_ID  INT           NOT NULL
        REFERENCES PASSENGER(Passenger_ID)
);
```

```
-- 4. RESERVATION
CREATE TABLE RESERVATION (
    Reservation_ID    INT         PRIMARY KEY,
    Reservation_Date  DATE        NOT NULL,
    Status            VARCHAR(20) NOT NULL DEFAULT 'Pending'
        CHECK (Status IN ('Confirmed','Pending','Cancelled')),
    Passenger_ID      INT         NOT NULL
        REFERENCES PASSENGER(Passenger_ID)
);
```

```sql
-- 5. AIRLINE
CREATE TABLE AIRLINE (
    Airline_ID    INT          PRIMARY KEY,
    Airline_Name  VARCHAR(100) NOT NULL UNIQUE
);

-- 6. AIRPORT
CREATE TABLE AIRPORT (
    Airport_ID  INT          PRIMARY KEY,
    Name        VARCHAR(150) NOT NULL,
    Location    VARCHAR(100) NOT NULL
);

-- 7. STAFF
CREATE TABLE STAFF (
    Staff_ID  INT         PRIMARY KEY,
    Name      VARCHAR(100) NOT NULL,
    Role      VARCHAR(50) NOT NULL
        CHECK (Role IN ('Pilot','Co-Pilot','Cabin Crew','Ground Staff','Manager'))
);
```

```sql
-- 8. FLIGHT
CREATE TABLE FLIGHT (
    Flight_ID      INT         PRIMARY KEY,
    Flight_Number  VARCHAR(10) NOT NULL UNIQUE,
    Departure_Time DATETIME    NOT NULL,
    Arrival_Time   DATETIME    NOT NULL,
    Airline_ID     INT         NOT NULL REFERENCES AIRLINE(Airline_ID),
    Airport_ID     INT         NOT NULL REFERENCES AIRPORT(Airport_ID),
    Staff_ID       INT         NOT NULL REFERENCES STAFF(Staff_ID),
    CHECK (Arrival_Time > Departure_Time)
);
```

```
-- 9. SCHEDULE
CREATE TABLE SCHEDULE (
    Schedule_ID  INT  PRIMARY KEY,
    Date         DATE NOT NULL,
    Time         TIME NOT NULL,
    Flight_ID    INT  NOT NULL REFERENCES FLIGHT(Flight_ID)
);

-- 10. SEAT
CREATE TABLE SEAT (
    Seat_ID      INT         PRIMARY KEY,
    Seat_Number  VARCHAR(5)  NOT NULL,
    Type         VARCHAR(20) NOT NULL
        CHECK (Type IN ('Window','Middle','Aisle')),
    Flight_ID    INT         NOT NULL REFERENCES FLIGHT(Flight_ID),
    UNIQUE (Seat_Number, Flight_ID)
);

-- 11. TICKET
CREATE TABLE TICKET (
    Ticket_ID       INT           PRIMARY KEY,
    Seat_Number     VARCHAR(5)    NOT NULL,
    Class           VARCHAR(20)   NOT NULL
        CHECK (Class IN ('Economy','Business','First')),
    Price           DECIMAL(10,2) NOT NULL CHECK (Price > 0),
    Reservation_ID  INT           NOT NULL REFERENCES RESERVATION(Reservation_ID),
    Seat_ID         INT           NOT NULL REFERENCES SEAT(Seat_ID),
    UNIQUE (Seat_ID)
);
```

3.2 Data Manipulation Language (DML) — Sample Data

Realistic sample data is inserted below. Each table contains 5–10 rows to enable meaningful analytical queries.

```
-- PASSENGER
INSERT INTO PASSENGER VALUES
(1,'Ahmed Al-Rashid','ahmed@email.com','0791234567','JO12345678'),
(2,'Sara Mohammed',  'sara@email.com', '0797654321','JO87654321'),
(3,'John Smith',     'john@email.com', '0798888888','US11223344'),
(4,'Layla Hassan',   'layla@email.com','0795555555','JO55667788'),
(5,'Carlos Rivera',  'carlos@email.com','0794444444','MX99887766'),
(6,'Yuki Tanaka',    'yuki@email.com', '0793333333','JP11112222'),
(7,'Fatima Al-Ali',  'fatima@email.com','0792222222','JO33445566');
```

```
-- AIRLINE
INSERT INTO AIRLINE VALUES
(1,'Royal Jordanian'),(2,'Emirates'),(3,'Qatar Airways'),
(4,'Turkish Airlines'),(5,'Lufthansa');

-- AIRPORT
INSERT INTO AIRPORT VALUES
(1,'Queen Alia International','Amman, Jordan'),
(2,'Dubai International',     'Dubai, UAE'),
(3,'Hamad International',     'Doha, Qatar'),
(4,'Ataturk International',   'Istanbul, Turkey'),
(5,'Frankfurt Airport',       'Frankfurt, Germany'),
(6,'Heathrow Airport',        'London, UK');
```

```
-- STAFF
INSERT INTO STAFF VALUES
(1,'Capt. Nasser Al-Omari','Pilot'),
(2,'Capt. Linda Baker',    'Co-Pilot'),
(3,'Omar Khalil',          'Cabin Crew'),
(4,'Rania Yousef',         'Manager'),
(5,'Hasan Al-Douri',       'Ground Staff'),
(6,'Emily Clarke',         'Cabin Crew');
```

```
-- FLIGHT
INSERT INTO FLIGHT VALUES
(1,'RJ101','2026-07-01 08:00','2026-07-01 11:30',1,1,1),
(2,'EK202','2026-07-02 14:00','2026-07-02 17:45',2,2,2),
(3,'QR303','2026-07-03 06:30','2026-07-03 09:00',3,3,3),
(4,'TK404','2026-07-04 22:00','2026-07-05 01:30',4,4,4),
(5,'LH505','2026-07-05 10:15','2026-07-05 14:00',5,5,5),
(6,'RJ106','2026-07-06 07:00','2026-07-06 10:45',1,6,1);
```

```
-- SCHEDULE
INSERT INTO SCHEDULE VALUES
(1,'2026-07-01','08:00',1),(2,'2026-07-02','14:00',2),
(3,'2026-07-03','06:30',3),(4,'2026-07-04','22:00',4),
(5,'2026-07-05','10:15',5),(6,'2026-07-06','07:00',6);

-- SEAT
INSERT INTO SEAT VALUES
(1,'1A','Window',1),(2,'1B','Middle',1),(3,'1C','Aisle',1),
(4,'2A','Window',2),(5,'2B','Middle',2),(6,'3A','Window',3),
(7,'4A','Aisle',4),(8,'5A','Window',5),(9,'6A','Aisle',6),(10,'6B','Middle',6);
```

```
-- RESERVATION
INSERT INTO RESERVATION VALUES
(1,'2026-06-01','Confirmed',1),(2,'2026-06-02','Confirmed',2),
(3,'2026-06-03','Pending', 3),(4,'2026-06-04','Cancelled', 4),
(5,'2026-06-05','Confirmed',5),(6,'2026-06-06','Confirmed',6),
(7,'2026-06-07','Pending', 7);

-- TICKET
INSERT INTO TICKET VALUES
(1,'1A','Business', 450.00,1,1),(2,'1B','Economy', 199.00,2,2),
(3,'1C','First',    900.00,3,3),(4,'2A','Economy', 210.00,5,4),
(5,'2B','Business', 480.00,6,5),(6,'3A','Economy', 175.00,7,6);
```

```
-- PAYMENT
INSERT INTO PAYMENT VALUES
(1,450.00,'2026-06-01','Credit Card',1),
(2,199.00,'2026-06-02','Online',     2),
(3,900.00,'2026-06-03','Debit Card',  3),
(4,210.00,'2026-06-05','Cash',        5),
(5,480.00,'2026-06-06','Credit Card', 6),
(6,175.00,'2026-06-07','Online',      7);

-- BAGGAGE
INSERT INTO BAGGAGE VALUES
(1,23.5,'Checked', 1),(2,7.0,'Carry-On',2),(3,30.0,'Oversized',3),
(4,20.0,'Checked', 4),(5,5.5,'Carry-On',5),(6,25.0,'Checked',  6),(7,15.0,'Checked',7);
```

3.3 Analytical SQL Queries

Query 1: INNER JOIN — Passenger Flight Bookings (3+ tables)

Retrieves each passenger's name alongside their reservation details and the flight number they booked. Joins PASSENGER, RESERVATION, TICKET, SEAT, and FLIGHT.

```
SELECT
    P.Name           AS Passenger,
    R.Reservation_ID,
    R.Status,
    T.Class,
    T.Price,
    F.Flight_Number,
    F.Departure_Time
FROM PASSENGER P
    JOIN RESERVATION R ON P.Passenger_ID  = R.Passenger_ID
    JOIN TICKET      T ON R.Reservation_ID = T.Reservation_ID
    JOIN SEAT        S ON T.Seat_ID        = S.Seat_ID
    JOIN FLIGHT      F ON S.Flight_ID      = F.Flight_ID
ORDER BY R.Reservation_ID;
```

Sample Output:

Passenger	Res. ID	Status	Class	Price	Flight	Departure
Ahmed Al-Rashid	1	Confirmed	Business	450.00	RJ101	2026-07-01 08:00
Sara Mohammed	2	Confirmed	Economy	199.00	RJ101	2026-07-01 08:00
John Smith	3	Pending	First	900.00	RJ101	2026-07-01 08:00
Carlos Rivera	5	Confirmed	Economy	210.00	EK202	2026-07-02 14:00
Yuki Tanaka	6	Confirmed	Business	480.00	EK202	2026-07-02 14:00
Fatima Al-Ali	7	Pending	Economy	175.00	QR303	2026-07-03 06:30

Query 2: LEFT OUTER JOIN — Passengers Without Tickets

Finds passengers who have a reservation but no ticket issued yet (unmatched data). Useful for identifying incomplete bookings.

```
SELECT
    P.Name           AS Passenger,
    R.Reservation_ID,
    R.Status,
    T.Ticket_ID,
    T.Class
FROM PASSENGER P
    LEFT JOIN RESERVATION R ON P.Passenger_ID  = R.Passenger_ID
    LEFT JOIN TICKET      T ON R.Reservation_ID = T.Reservation_ID
WHERE T.Ticket_ID IS NULL
ORDER BY P.Name;
```

Sample Output:

Passenger	Reservation ID	Status	Ticket ID	Class
Layla Hassan	4	Cancelled	NULL	NULL

Query 3: Aggregate — Revenue by Flight Class (GROUP BY / HAVING)

Calculates total revenue and average ticket price per flight and class, filtered to show only class-flight combinations earning more than $300.

```
SELECT
    F.Flight_Number,
    T.Class,
    COUNT(T.Ticket_ID)  AS Tickets_Sold,
    SUM(T.Price)        AS Total_Revenue,
    AVG(T.Price)        AS Avg_Price
FROM TICKET T
    JOIN SEAT   S ON T.Seat_ID  = S.Seat_ID
    JOIN FLIGHT F ON S.Flight_ID = F.Flight_ID
GROUP BY F.Flight_Number, T.Class
HAVING SUM(T.Price) > 300
ORDER BY Total_Revenue DESC;
```

Sample Output:

Flight	Class	Tickets Sold	Total Revenue ($)	Avg Price ($)
RJ101	First	1	900.00	900.00
EK202	Business	1	480.00	480.00
RJ101	Business	1	450.00	450.00
EK202	Economy	1	210.00	210.00

Query 4: CTE — Top-Spending Passengers

Uses a Common Table Expression to rank passengers by their total spending on tickets, then returns the top spenders along with their booking count.

```
WITH PassengerSpend AS (
    SELECT
        P.Passenger_ID,
        P.Name,
        COUNT(T.Ticket_ID)  AS Total_Bookings,
        SUM(T.Price)        AS Total_Spent
    FROM PASSENGER P
        JOIN RESERVATION R ON P.Passenger_ID  = R.Passenger_ID
        JOIN TICKET      T ON R.Reservation_ID = T.Reservation_ID
    GROUP BY P.Passenger_ID, P.Name
)
SELECT
    Name,
    Total_Bookings,
    Total_Spent,
    RANK() OVER (ORDER BY Total_Spent DESC) AS Spend_Rank
FROM PassengerSpend
ORDER BY Total_Spent DESC;
```

Sample Output:

Passenger	Total Bookings	Total Spent ($)	Rank
John Smith	1	900.00	1
Yuki Tanaka	1	480.00	2
Ahmed Al-Rashid	1	450.00	3
Carlos Rivera	1	210.00	4
Sara Mohammed	1	199.00	5
Fatima Al-Ali	1	175.00	6

PHASE 0 · PROJECT DELIVERABLES CHECKLIST · All Phases Summary

Ph 1	✔	**Project Description**	5–10 line overview of the Airlines Reservation System
Ph 1	✔	**Business Rules**	11 rules covering all entities and relationships
Ph 1	✔	**ERD Diagram**	Chen notation diagram with 11 entities, attributes, PKs, relationships
Ph 1	✔	**Entity-Attribute List**	Complete attribute list per entity
Ph 1	✔	**Relationship Table**	Cardinalities and descriptions for all 9 relationships
Ph 2	✔	**Relational Schema**	11 tables with PKs and FKs in standard notation
Ph 2	✔	**Normalization (3NF)**	FDs listed and 3NF verified for all 11 tables
Ph 3	✔	**DDL Script**	CREATE TABLE for all 11 tables with constraints
Ph 3	✔	**DML Script**	5–10 rows per table, realistic data
Ph 3	✔	**Query 1 – INNER JOIN**	3-table join: PASSENGER, RESERVATION, TICKET, SEAT, FLIGHT
Ph 3	✔	**Query 2 – OUTER JOIN**	LEFT JOIN to find passengers without tickets
Ph 3	✔	**Query 3 – Aggregate**	GROUP BY Class/Flight with SUM, AVG, COUNT and HAVING
Ph 3	✔	**Query 4 – CTE**	Common Table Expression ranking passengers by spend